

Feature Guide

Contents:

Evolution 2

- [Audit Logs](#)
- [Session Timeouts](#)
- [MFA](#)
- [Program Description Rich Text](#)

Evolution 1

- [GUI Editable Home Page](#)

Audit Logs

Design

Logins and modifications to data made on the system are recorded in a WORM log file. Entries are tagged with timestamps, IP addresses, api requests made, and the user accounts that undertook them. Access to these logs are restricted to restrict this access to people with access to code base or super users who can access the Django admin panel.

Back End

settings.py: Add custom middleware for logging and log file specifications

```
MIDDLEWARE = [
    'audit_logging.audit_middleware.AuditLoggingMiddleware',
]

LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "audit_format": {
            "format": "%(asctime)s %(levelname)s - %(message)s",
            "datefmt": "%Y-%m-%d %H:%M:%S",
        },
    },
    "handlers": {
        "audit_log": {
            "level": "INFO",
            "class": "logging.handlers.RotatingFileHandler",
            "filename": os.path.join(os.path.dirname(__file__), "audit.log"),
            "maxBytes": 1024 * 1024 * 10, # 10MB per file
            "backupCount": 10,
            "formatter": "audit_format",
        },
    },
    "loggers": {
        "audit_logger": {
            "handlers": ["audit_log"],
            "level": "INFO",
            "propagate": False,
        },
    },
}
```

audit_middleware.py

Summary

- **log_request(request)** – Extracts and logs request details such as method, path, user, IP, headers, and body.
- **AuditLoggingMiddleware Class** – Main middleware that handles logging.
 - **__init__(self, get_response)** – Initializes middleware, ensures the log file exists
 - **__call__(self, request)** – Intercepts requests, logs relevant details, and determines if special logging is required (e.g., login, SSO).
 - **get_client_ip(self, request)** – Extracts the real IP address from request headers.
 - **get_request_data(self, request)** – Extracts request data, masking sensitive fields (e.g., passwords, tokens).
 - **get_username_from_request(self, request)** – Retrieves the username from request data for login attempts.

Command line permissions on server

```
sudo chown www-data:www-data /home/jc870/abroad-hub/backend/backend/audit.log
```

Logging Behavior:

- Logs all user actions, particularly modifications (**POST**, **PUT**, **DELETE**, **PATCH**).
- Masks sensitive fields in request data.
- Detects login or SSO-related requests and logs them separately.
- Enforces secure log storage with WORM permissions.

audit.log

- Prints logs for logins and modifications in the following format:

```
[2025-02-27 04:11:29] INFO - [2025-02-27T04:11:29.484693] IP: 127.0.0.1, User: janedoe,  
Method: POST, Path: /api/token/, Data: {"username": "janedoe", "password": "*****"}
```

Front End

- Superusers, via the django admin panel, can view the logs by visiting **/admin/audit-logs/**

Session Timeouts

Design

settings.py: set the inactivity limit and refresh token timeout for JWT tokens in the backend

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15), # Inactivity timeout
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1), # Hard session expiration
    "ROTATE_REFRESH_TOKENS": False, # Prevent extending the session past 24 hours
    "BLACKLIST_AFTER_ROTATION": True, # Blacklist old refresh tokens
}
```

ProtectedRoute.tsx: ProtectedRoute wraps every page. If the limits are violated the user is logged out and redirected to the login page. If a user's session times out they are informed that they are being forced to logout.

```
// Check inactivity timeout (15 min)
if (lastActivity && now - lastActivity > INACTIVITY_LIMIT) {
    alert("Session expired due to inactivity. Please log in again.");
    forceLogout();
    return;
}

// Check session timeout (24 hours)
if (loginTime && now - loginTime > SESSION_LIMIT) {
    alert("Your session has expired. Please log in again.");
    forceLogout();
    return;
}

...
const forceLogout = () => {
    setSessionTimedOut(true);
    router.replace("/logout");
};

...
if (isSessionTimeOut) {
    return <div>Session timeout. Force Logout...</div>;
}
```

Benefits

Setting JWT token limits helps enhance security, prevent unauthorized access, and provides a step towards security standards required by HIPAA and FERPA.

Optional MFA with Google Authenticator for Local Users

Design

Database Configuration

Each user is given a `use_mfa` boolean field. In order to require mfa on login, the `use_mfa` must be true and the user must not be an sso user.

APIs

`api/mfa-settings/`

- updates the user's `use_mfa` field when they enable/disable mfa in profile settings

`api/verify-otp/`

- Verifies that the user's entered OTP matches the one stored for the session in the backend

`api/token/`

- Uses a custom token obtain pair view that sends back the access/refresh tokens or an OTP url if mfa is required for that user.

Back End

settings.py: add necessary installed apps, middleware, and email address variables

```
INSTALLED_APPS = [
    . . .
    'django_otp',
    'django_otp.plugins.otp_totp',  # TOTP plugin for Google Authenticator
]

MIDDLEWARE = [
    . . .
    'django_otp.middleware.OTPMiddleware',
]
```

Front End

MFA settings component is reachable from a user's profile and allows them to enable/disable mfa.

Props

- **User:** current user info to get default mfaEnable value
- **Route:** where router pushes page onSubmit

```
<MFASettings user={current_user} route={route_to_profile}/>
```

OTP verification component displays a qr code to scan with the Google Authenticator App and a form to submit a 6 digit OTP.

Props

- **onVerificationSuccess:** action to be taken if OTP is correct which involves logging in and directing to the correct home page.
- **otpUrl:** otpUrl to be configured into a qr code for the user

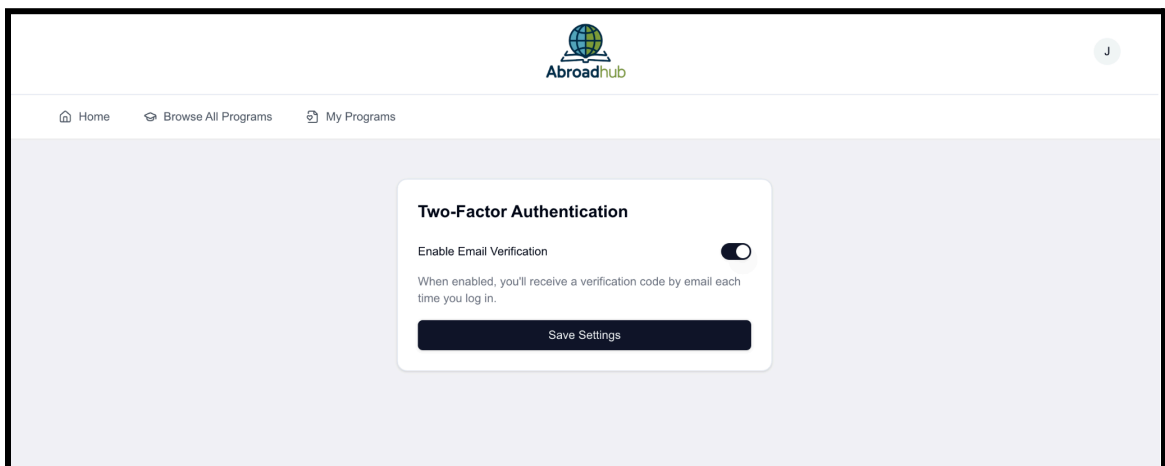
```
<OTPVerification onVerificationSuccess={handler} otpUrl={otpUrl}/>
```

Benefits

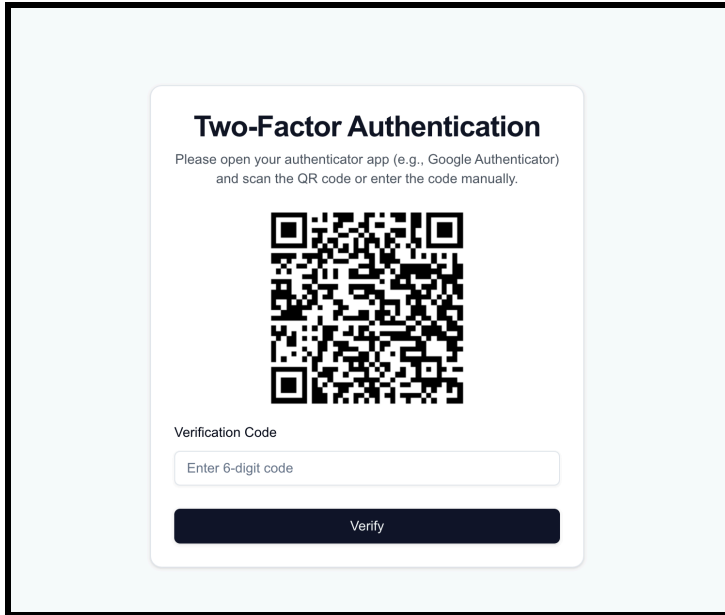
Provides security to local user accounts by requiring another step before login is accepted. Using email is convenient because local users already provide email on signup and it is more secure than using SMS.

Walkthrough

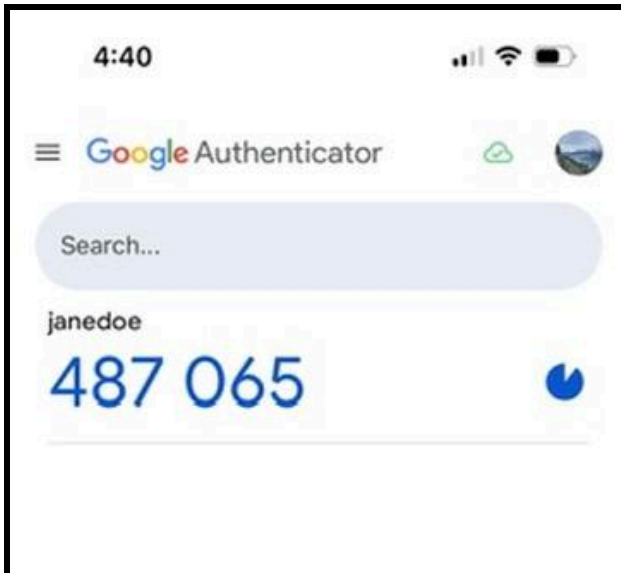
- Log in as a local user and navigate to profile. Click MFA Settings
- Enable MFA and save settings



- Next time you login you will receive a qr code to scan and a form to enter the 6 digit password shown on the app. Enter your code and click Verify



- When you open the Google Authenticator app there is a plus sign in the lower right hand corner. Click the plus sign and select scan qr code. Once you scan the qr code the app will display 6 digit codes that refresh on a timer. **You never have to scan the qr code again after you register it the first time with the app**, your codes will continuously be regenerated and you can just open the app to find the current valid code on your next login.



Rich Text for Program Descriptions

Design

Database Configuration

The description is stored in the database as a field in the program table:

Table: **Program**

description: Textfield

Front End

The program form displays the rich text tip tap editor as the form input for descriptions

```
<label
  htmlFor="description"
  className="block text-gray-700 font-semibold mb-2"
>
  Description:
</label>
<Controller
  name="description"
  control={control}
  defaultValue={programProps.descriptionDefault ?? ""}
  render={({ field }) => (
    <Tiptap
      editorContent={field.value}
      onChange={field.onChange}
      isEditing={true}
    />
  )}
/>
```

The description is rendered inside **dangerouslySetInnerHTML** when benign displayed, ensuring that the page displays formatted rich text.

```
<div>
  <strong>Description:</strong>
  <div
```



```
        dangerouslySetInnerHTML={{
          __html: application.program.description,
        }}
      />
    </div>
```

Benefits

The rich text program description offers administrators the opportunity to make programs visibly more enticing and emphasize important information. It also allows desired paragraph formatting to be kept.

Walkthrough

- Log in as administrator and navigate to the Programs Overview Page on the top navbar
- Locate the create program button
- In the description box format the text as you would like it displayed and use the toolbox headers for text customization. When finished, fill out the rest of the program and submit

Semester:

Fall

Description:

B / **H₂**  

GUI-editable home page

Design

Database Configuration

The content is stored in the database with the following table configuration:

Table: **Content**

content: Textfield with default value

save: overridden to enforce singleton constraint

API

/content/

Method: GET, PUT

Purpose: Retrieve and overwrite string content related to the home page

Request: Content data

Response: Content data

Front End

The banner content is rendered inside **dangerouslySetInnerHTML**, ensuring that the page displays formatted rich text.

```
<div dangerouslySetInnerHTML={{ __html: content }} />
```

Admin can update content using a TipTap rich text editor component

```
{user && !is_student ? (  
  <Tiptap editorContent={content} onChange={setContent} />  
) : null}
```

Benefits

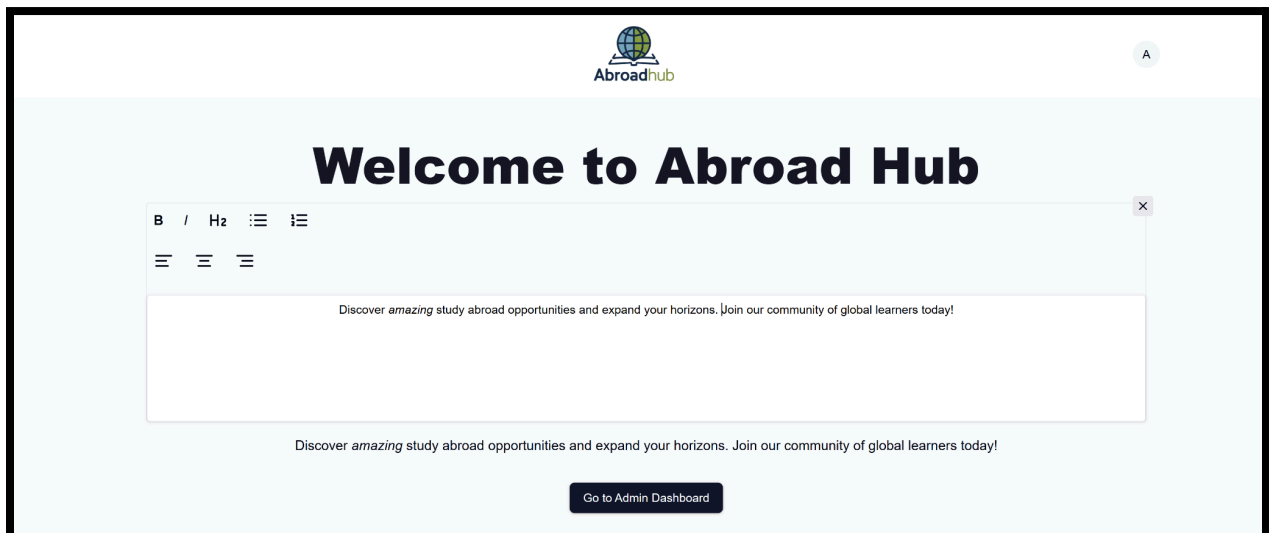
The GUI-editable home page offers administrators fast communication and easy updates. They are able to post content for students quickly and without needing technical knowledge. This is very useful as the study abroad program may have urgent or important matters that need to be conveyed to users. Additionally, offering rich text improves the user interface and provides administrators more tools to communicate effectively. They can emphasize with bold or italics to grab students' attention, or create bulleted lists to provide important information succinctly.

Walkthrough

- Log in as administrator and navigate to the home page
- Locate the edit button to the right of the edit-able banner



- Use the rich text editor to create the new banner



- Click x when complete and view the new banner from the homepage



- Optionally: log in as a student to confirm that students can see the updated home page

